



UNIDADE III

Engenharia de Software Ágil Aplicada

Prof. Me. Edson Moreno

Introdução

- A evolução do projeto e construção de software depende da integração entre gestão de projetos, engenharia de software e arquitetura de sistemas.
- O PMBOK® convive com metodologias ágeis, promovendo maior flexibilidade, entregas incrementais e foco no valor ao cliente.
- O SWEBOK® e padrões como ISO, IEEE e CMMI orientam práticas técnicas voltadas à qualidade, padronização e evolução contínua dos sistemas.
- Em complemento, a arquitetura de sistemas fornece a estrutura que apoia arquiteturas distribuídas, microsserviços, APIs e práticas como CI/CD e DevSecOps, sustentando soluções adaptáveis em ecossistemas digitais dinâmicos.

5. Projeto e construção do software

- O projeto e a construção do software representam o momento em que a visão conceitual se transforma em produto funcional, exigindo sinergia entre planejamento, modelagem, arquitetura e codificação orientada à qualidade.
- No desenvolvimento do projeto de software, são estruturados diversos processos sistemáticos de acompanhamento e monitoramento que abrangem todas as suas fases de concepção, implementação e implantação, incluindo também suporte e manutenção.

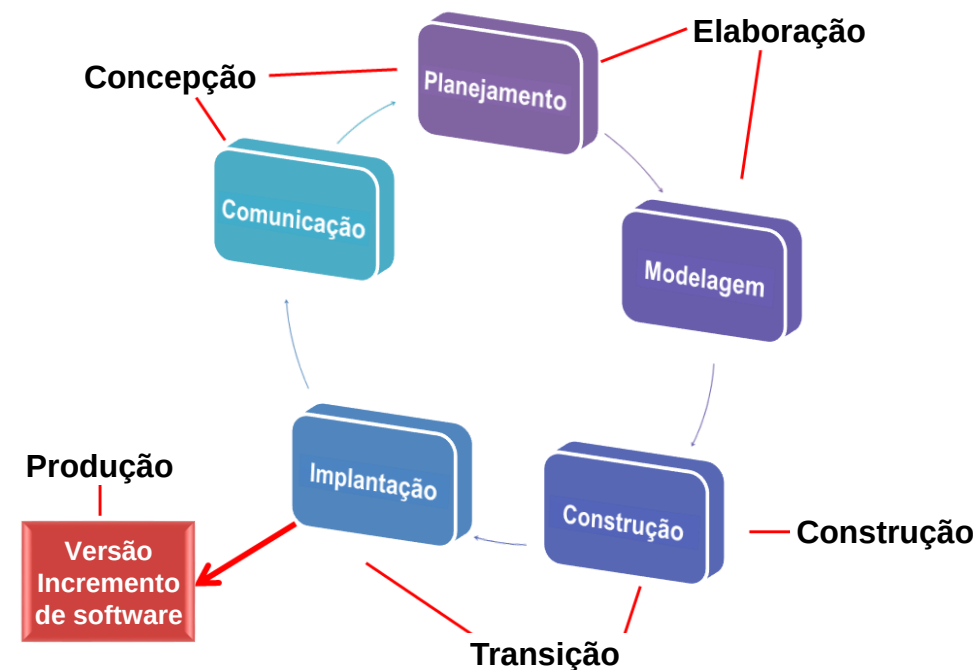


Figura - Fases do processo unificado.

Fonte: Pressman (2016).

Nesta seção, serão abordados:

Gerenciamento do projeto de software

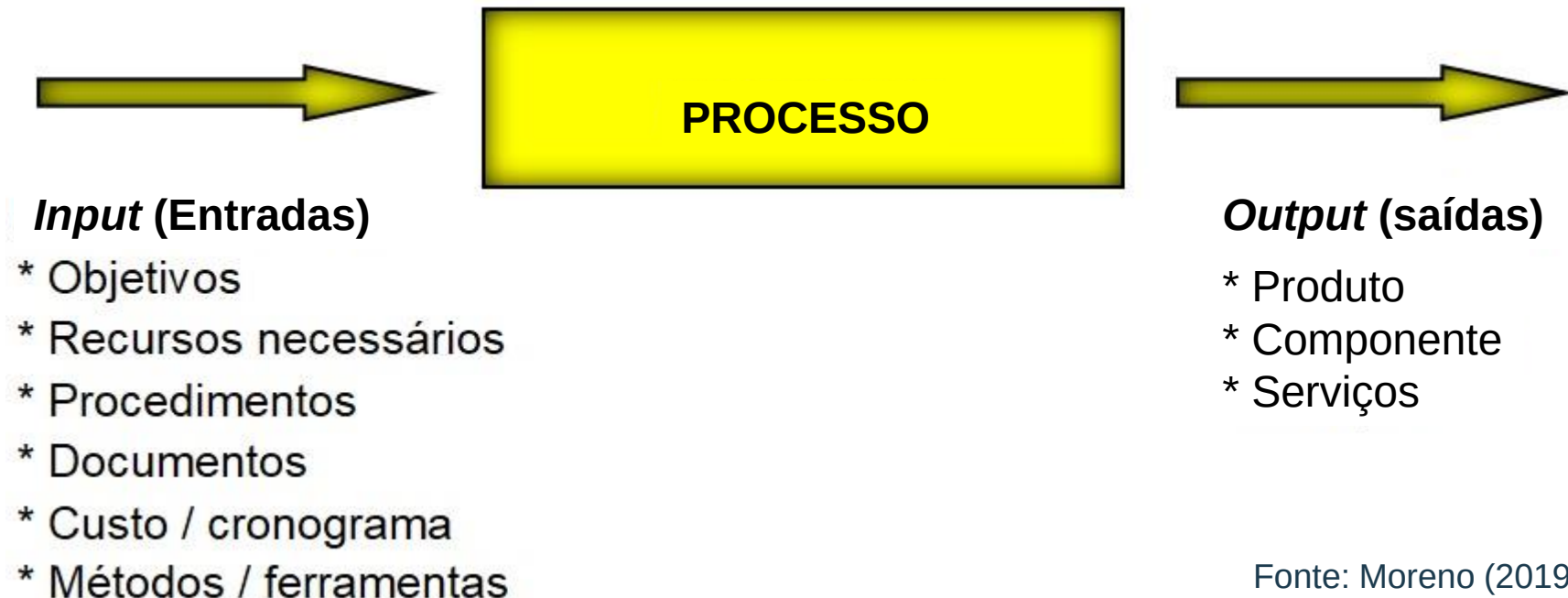
Guia do conhecimento em gerenciamento de projetos PMBOK®

Guia de referência para engenharia de software SWEBOK®

Arquiteturas de sistemas: Uma Abordagem Integrada entre PMBOK® e SWEBOK®

Gerenciamento do projeto de software

- O ciclo de vida do projeto é gerenciado por meio da execução de uma série de atividades de gerenciamento de projeto conhecidas como processo de gerenciamento de projetos.
- Um processo é uma sequência de fatos, atividades ou operações que apresentam certa unidade e/ou que se reproduzem com certa regularidade.
- O processo deve ser delimitado, definindo seu início e fim. Os pontos de início (*input*) e fim (*output*) são marcos de referência (milestones), úteis para o acompanhamento do processo.
 - Os marcos de referência servem para controlar, medir, avaliar, projetar e melhorar o processo.



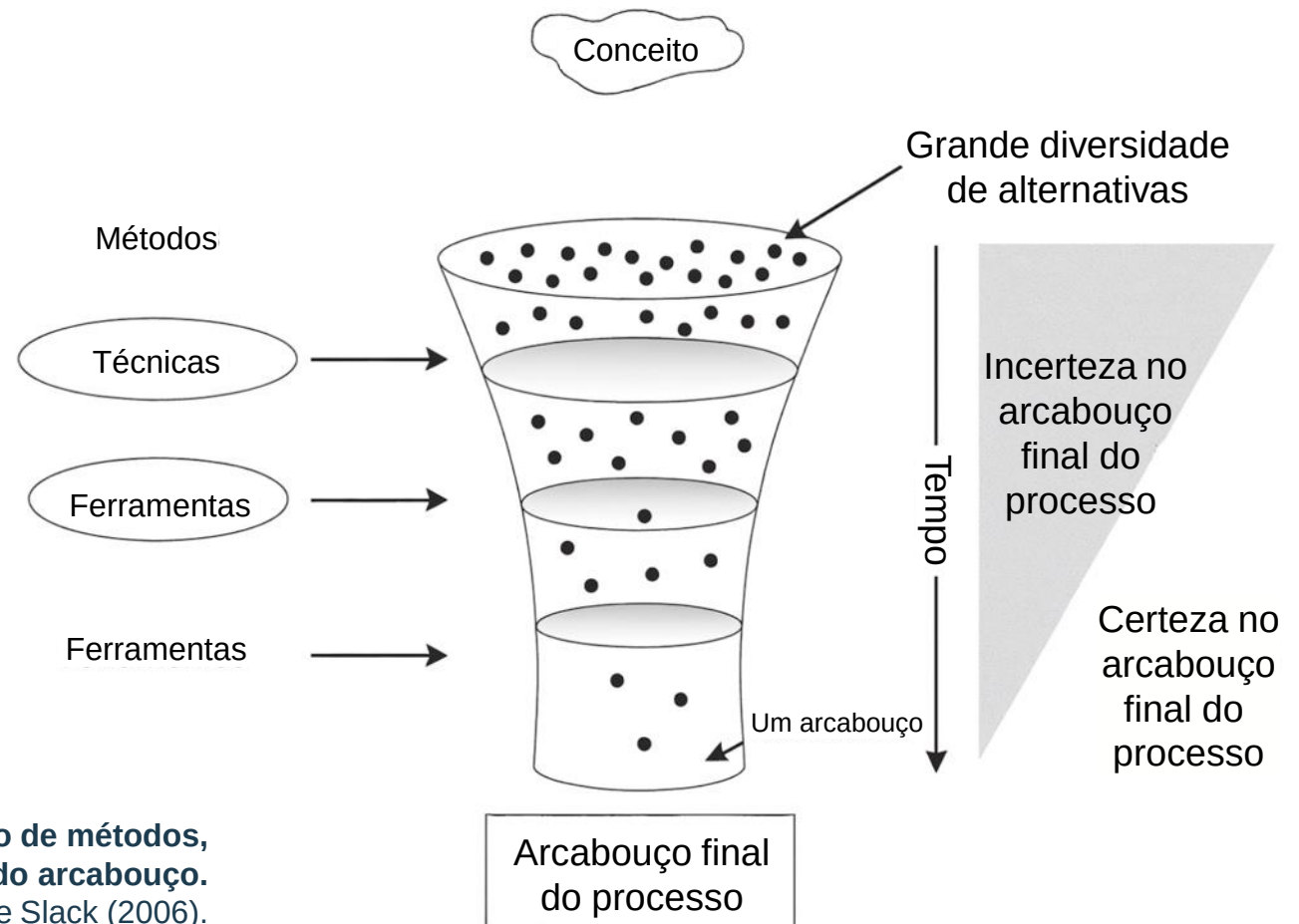
Arcabouço do processo de software

- Um arcabouço de processo (ou framework de processo) é uma estrutura de suporte que fornece uma base comum e organizada para a execução de atividades.
- É o alicerce que garante que todos falem a mesma língua e sigam um fluxo lógico, mantendo a flexibilidade para inovar.
- Fazem parte da composição do arcabouço: componentes, métodos, técnicas e ferramentas.
- Esse arcabouço estabelece a base metodológica sobre a qual se estruturam as fases de planejamento, *design*, implementação e verificação, permitindo que o desenvolvimento ocorra de forma adaptativa e incremental.
- Exemplo de arcabouço do processo com Métodos, Ferramentas e Técnicas (MF&T).

MÉTODOS	FERRAMENTAS	TÉCNICAS
Cascata Incremental Scrum Kanban DevOps ISO 12207 CMMI MPS.BR	Astah Jira Android Studio Eclipse Visual Studio GitHub Microsoft Project Google Cloud Plataforma	User Stories Prototipagem Modelagem UML Arquiteturas Testes Qualidade, PDCA Matriz SWOT Versionamento

Análise e definição do arcabouço

- A definição do arcabouço segue um fluxo iterativo em que, durante a análise do contexto do projeto, são filtradas as alternativas de métodos, técnicas e ferramentas menos relevantes, mantendo-se apenas aquelas que se mostram essenciais para alcançar os objetivos do sistema.
- A etapa de abstração e modelagem do arcabouço permite representar com precisão componentes metodológicos e comportamentos esperados do processo, resultando em uma estrutura consistente, alinhada à qualidade do produto de software e à efetividade da engenharia de desenvolvimento.



Iteração para seleção de métodos,
ferramentas e técnicas do arcabouço.
Fonte: Moreno (2025), adaptado de Slack (2006).

Guia do Conhecimento em Gerenciamento de Projetos PMBOK®

- O bom projeto viabiliza o uso do sistema por um longo período de tempo, dispondo de recursos que permitam adaptá-lo a novas mudanças, correções e integração a novos ambientes operacionais.
- O Guia PMBOK® é uma base sobre a qual as organizações podem criar metodologias, políticas, procedimentos, regras, ferramentas e técnicas e fases do ciclo de vida necessários para a prática do gerenciamento de projetos (PMBOK, 2017).



Fonte: ÁVILA, Márcio. *PMBOK e Gerenciamento de Projetos*.
Disponível em: <http://www.mhavila.com.br/topicos/gestao/pmbok.html>,
18/09/2022. Acesso em: 30 jan. 2026.



Componentes-chave do guia PMBOK®

- O ciclo de vida do projeto pelo PMBOK® percorre as fases: Início do Projeto; Organização e preparação; Execução do trabalho; Término do projeto.
- O gerenciamento de projetos permite ter uma abordagem estruturada e sistemática para planejar, executar e controlar projetos de forma eficiente.

A estrutura do Guia PMBOK® possui diversos componentes-chave que são mostrados no quadro abaixo:

Componentes-Chave do Guia PMBOK®

Ciclo de vida do projeto

Fase do projeto

Revisão de fase

Grupo de processos de gerenciamento de projetos

Processo de gerenciamento de projetos

Área de conhecimento em gerenciamento de projetos

Guia de Referência para Engenharia de Software SWEBOK®

- O SWEBOK® é a referência global para padronização e prática profissional (IEEE Computer Society). É um guia que consolida o conhecimento essencial da engenharia de software, definindo o escopo da disciplina e padronizando termos e princípios.
 - Propósito – Definir o que um engenheiro de software deve dominar, orientando currículos, certificações e a prática profissional.
 - Diferencial – Não dita metodologias ("como fazer"), mas estabelece os fundamentos necessários ("o que conhecer").

O guia SWEBOK® é organizado em 18 Áreas de Conhecimento (KAs – v4.0), distribuídas nas seguintes categorias:

- Ciclo de Vida: Requisitos, Arquitetura, Projeto, Construção, Teste, Operações e Manutenção.
- Gestão e Suporte: Gerenciamento de Configuração, Gerenciamento de Engenharia, Processo, Modelos e Métodos, Qualidade e Segurança.
- Profissional e Econômico: Prática Profissional e Economia da Engenharia de Software.
- Fundamentos: Bases de Computação, Matemática e Engenharia.

Interatividade

Sobre o guia SWEBOK®, assinale a alternativa correta:

- a) Consolida fundamentos da engenharia de software como referência global.
- b) Define metodologias prescritivas para o desenvolvimento de software.
- c) Estabelece ferramentas obrigatórias para projetos de software.
- d) Foca exclusivamente na gestão ágil de projetos de software.
- e) Substitui normas ISO e modelos de maturidade de processo.

Resposta

Sobre o guia SWEBOK®, assinale a alternativa correta:

- a) **Consolida fundamentos da engenharia de software como referência global.**
- b) Define metodologias prescritivas para o desenvolvimento de software.
- c) Estabelece ferramentas obrigatórias para projetos de software.
- d) Foca exclusivamente na gestão ágil de projetos de software.
- e) Substitui normas ISO e modelos de maturidade de processo.

Integrando PMBOK® (Gerenciamento) e SWEBOK® (Execução Técnica)

1. A Visão da Gestão: PMBOK® (O "Como Gerenciar")

A arquitetura é tratada como um artefato-chave que impacta as restrições do projeto:

- Integração – O "plano mestre" que une equipes de back-end, front-end e banco de dados.
- Escopo – Um produto de entrega (deliverable) essencial para o planejamento.
- Qualidade – Onde se definem escalabilidade e segurança, validadas por processos de gestão.
- Riscos – Identifica gargalos técnicos (ex.: capacidade de CPU/Rede) antes que se tornem crises.

2. A Visão Técnica: SWEBOK® (O "O Que e Como Fazer")

A arquitetura é definida no nível técnico mais profundo, dentro da área de software design:

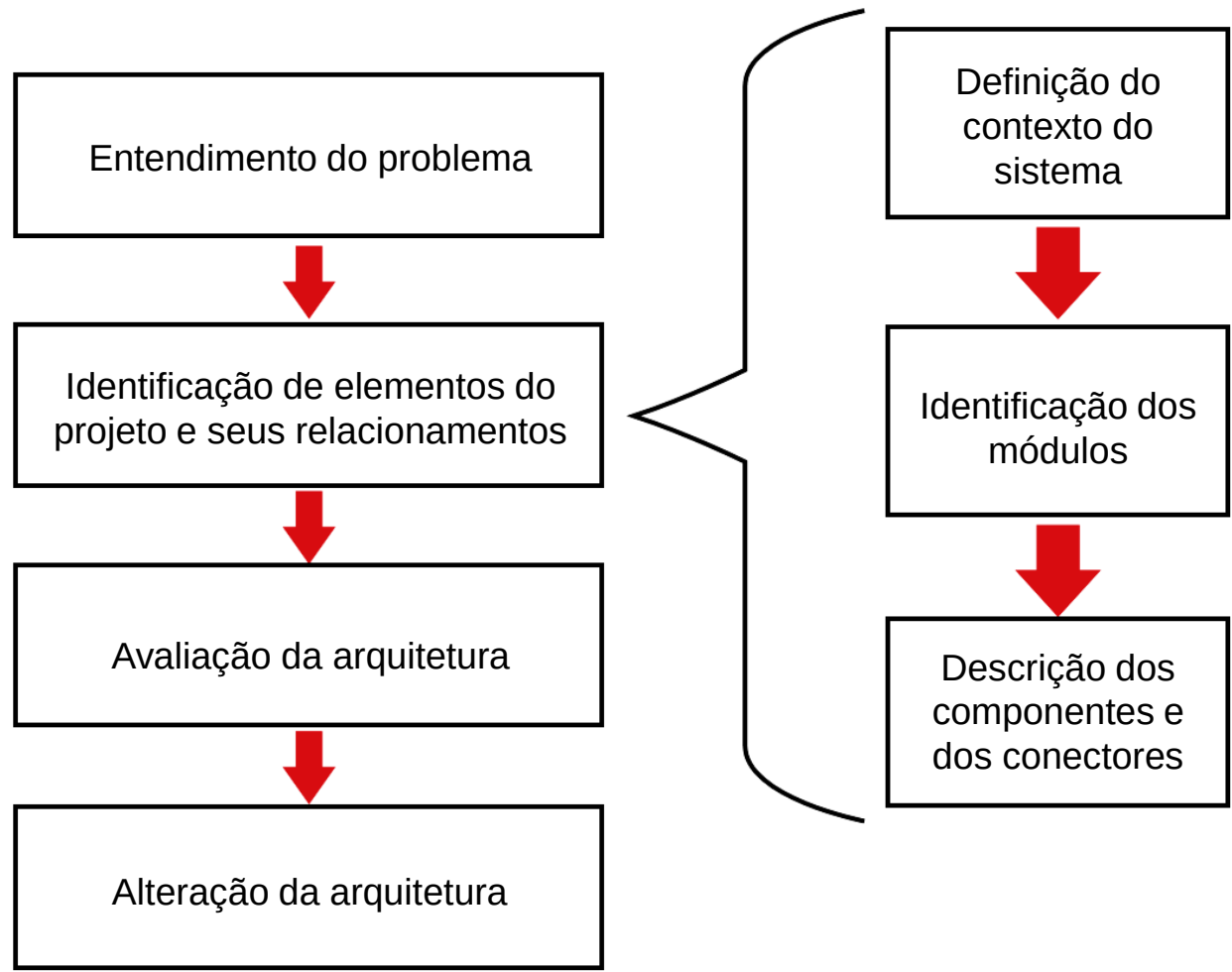
- Definição de Estrutura – Análise de requisitos para criar componentes, interfaces e relações.
- Estilos e Padrões – Escolha de modelos (Microsserviços, Camadas, Cliente-Servidor).
- Atributos de Qualidade – Foco técnico em manutenibilidade, desempenho e robustez.

Arquitetura de sistemas

Arquiteturas de sistemas são estruturas organizacionais que definem como os componentes de um sistema interagem, se comunicam e funcionam em conjunto para atender aos requisitos de negócio e técnicos.

- Ela funciona como um plano mestre que orienta o desenvolvimento, integração e evolução do sistema ao longo do tempo.
- É uma visão de alto nível, representando um mapa estratégico além do código-fonte, permitindo uma melhor abstração do sistema, de forma que a equipe entenda como as grandes peças se encaixam e interagem.

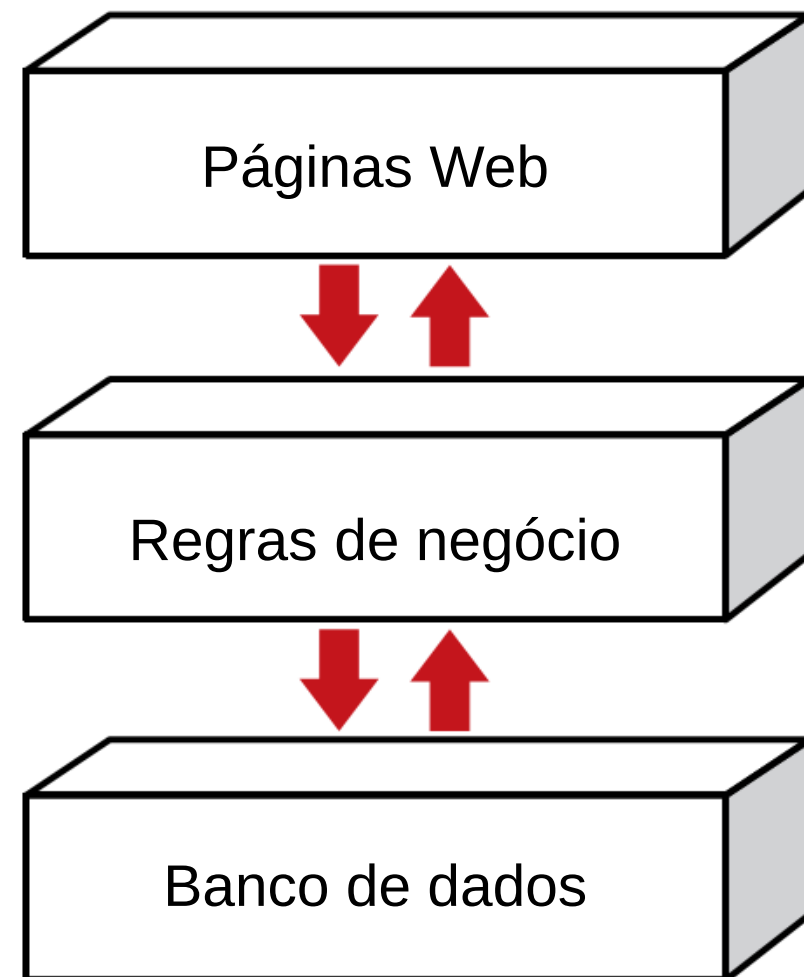
Na figura, estão as atividades praticadas no projeto arquitetural:



Elementos do sistema na arquitetura em camadas

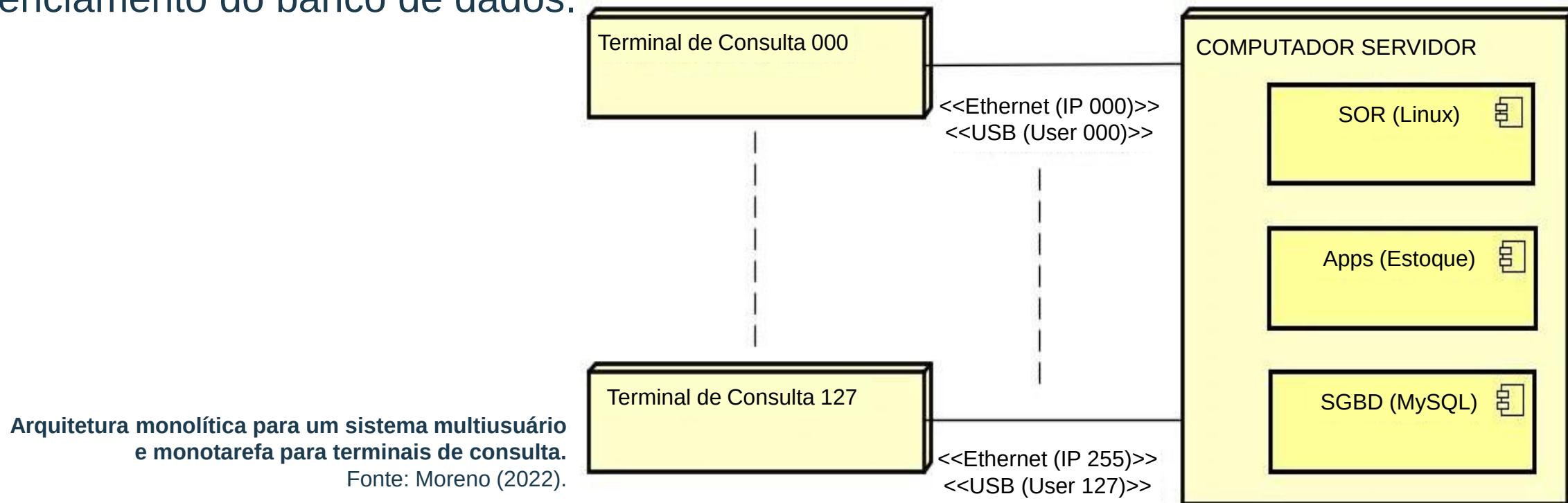
Principais elementos da arquitetura de sistemas:

- Camadas – Separação lógica entre apresentação, lógica de negócio e integração (ou dados), com foco no negócio.
 - Serviços – Funcionalidades específicas que o sistema de software oferece.
 - Interfaces – Pontos de interação entre componentes ou com o usuário.
 - Protocolos de comunicação – Regras, normalmente de rede de computadores para troca de dados entre os módulos do sistema de software.
 - Padrões de design – Soluções reutilizáveis para problemas recorrentes.
-
- Ao lado, vemos um modelo genérico de um ambiente operacional em que se apresenta como um sistema em camadas, no qual cada “caixa” corresponde a uma camada.



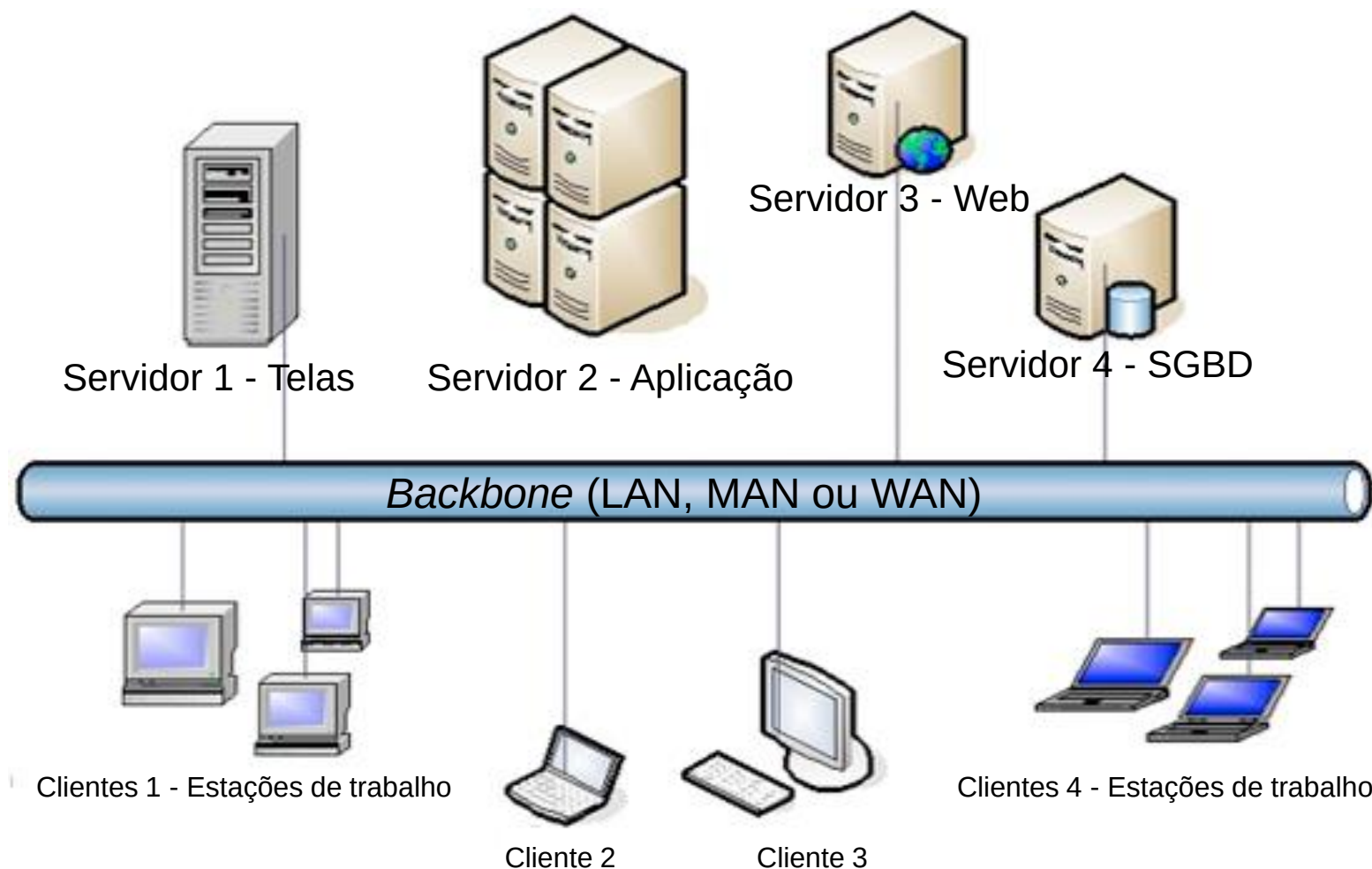
Arquitetura de sistemas monolíticos

- Na arquitetura monolítica, todos os componentes são integrados em um único bloco, servindo como base para a maioria dos sistemas no início da computação moderna, em que a simplicidade de implantação em uma única unidade era mais importante que a escalabilidade.
- Análise da arquitetura – A figura apresenta um exemplo de arquitetura monolítica para atendimento de terminais como PDV, leitores de códigos de barras, controle de catracas, sinalizadores e outros. Observe que é no “computador servidor” que é executada a aplicação e o gerenciamento do banco de dados.



Arquitetura cliente-servidor

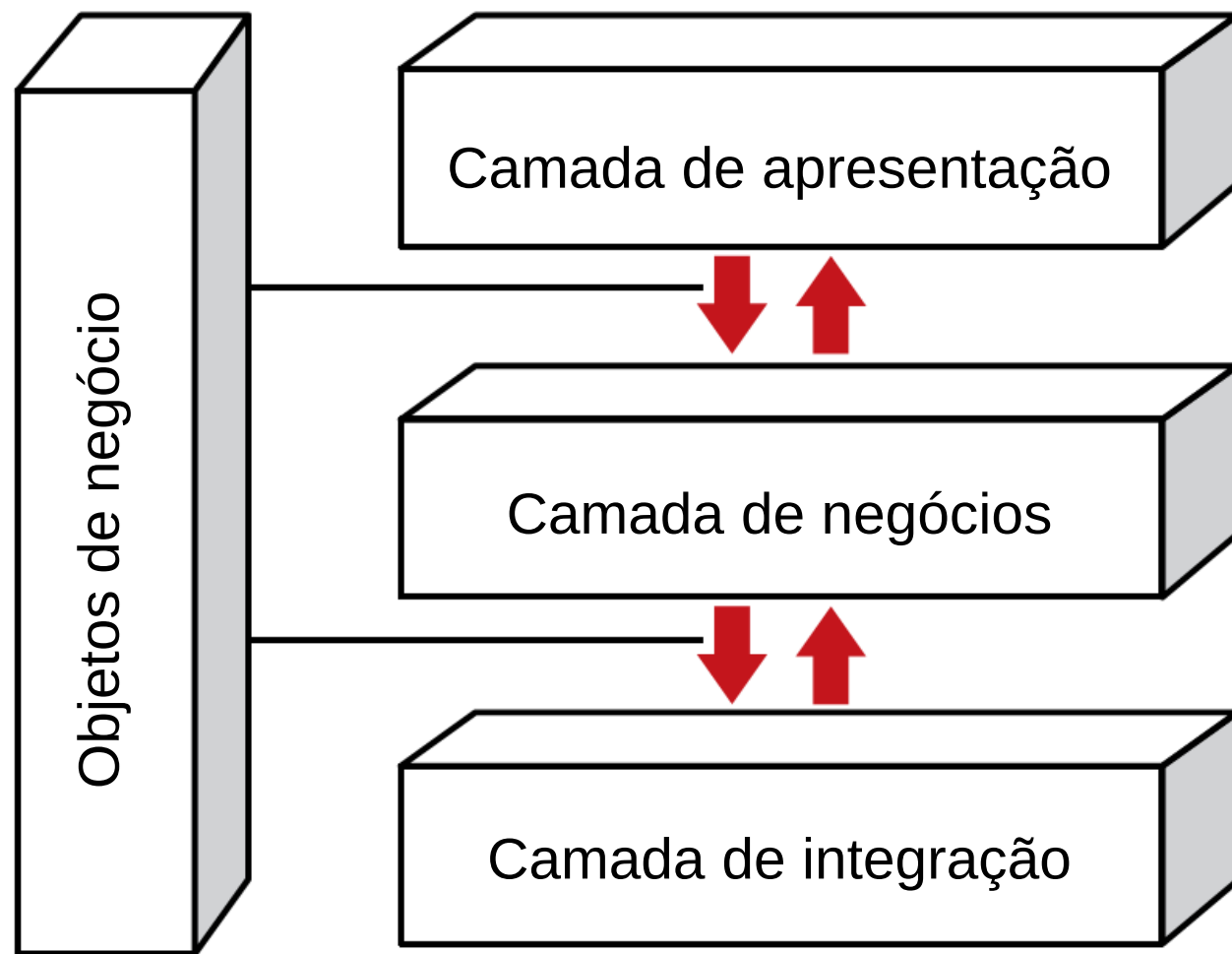
- Na arquitetura cliente-servidor, é apresentada a separação entre quem solicita e quem fornece os serviços, estabelecendo o primeiro grande padrão de distribuição básica, separando a apresentação (cliente) da lógica e dos dados (servidor).



Arquitetura em camadas

- A **arquitetura em três camadas** é um padrão estabelecido na construção de sistemas.

Camada	Definição
Apresentação	Classes responsáveis pela interação com o usuário.
Negócios (ou Aplicação)	Classes responsáveis pela execução das regras de negócio.
Integração	Classes responsáveis pela integração das tecnologias da informação. Por exemplo: SGBD.



Arquitetura de microsserviços – Sistemas distribuídos e escalabilidade

O Que Define os Microsserviços?

- Evolução do SOA (Arquitetura orientada a serviços) – Serviços menores, independentes e totalmente autônomos.
- Autonomia de Dados – Cada serviço possui seu próprio banco de dados e módulos de implantação.
- Comunicação Leve – Interação via protocolos simples como HTTP/REST ou mensageria.

Principais Diferenciais

- Escalabilidade Individual – Permite crescer apenas a parte do sistema que sofre maior demanda (comum em e-commerces).
- Alinhamento com Sistemas Distribuídos – É o modelo atual que melhor aplica os princípios de distribuição.
- Foco em Disponibilidade – A falha em um serviço não derruba necessariamente o sistema inteiro.
- Resumo Conceitual – Uma abstração de alto nível focada em velocidade de entrega, em que o software é uma coleção de pequenos serviços independentes que se comunicam entre si.

Interatividade

A arquitetura de microsserviços é amplamente utilizada em sistemas distribuídos modernos por suas características estruturais e operacionais. Considerando seus princípios e diferenciais, assinale a alternativa que melhor define essa abordagem arquitetural:

- a) Abordagem que exige forte acoplamento entre módulos para garantir desempenho.
- b) Arquitetura que prioriza o compartilhamento de dados para reduzir redundâncias.
- c) Estrutura composta por serviços independentes, com autonomia e comunicação leve.
- d) Modelo arquitetural baseado na centralização da lógica e dos dados em um único componente.
- e) Modelo orientado à execução sequencial de funcionalidades integradas.

Resposta

A arquitetura de microsserviços é amplamente utilizada em sistemas distribuídos modernos por suas características estruturais e operacionais. Considerando seus princípios e diferenciais, assinale a alternativa que melhor define essa abordagem arquitetural:

- a) Abordagem que exige forte acoplamento entre módulos para garantir desempenho.
- b) Arquitetura que prioriza o compartilhamento de dados para reduzir redundâncias.
- c) Estrutura composta por serviços independentes, com autonomia e comunicação leve.
- d) Modelo arquitetural baseado na centralização da lógica e dos dados em um único componente.
- e) Modelo orientado à execução sequencial de funcionalidades integradas.

6. Ecossistemas digitais e engenharia de software conectada

- O desenvolvimento de software atual ocorre em um cenário cada vez mais interconectado, dinâmico e orientado a serviços. É um cenário de criação e evolução tecnológica, no qual múltiplas plataformas, organizações, usuários e tecnologias interagem de forma contínua.

Ecossistemas Digitais

- Interconectividade – O software não é mais um produto isolado, mas um componente integrado a uma rede de APIs, nuvem, bancos de dados e dispositivos.
- Ambiente vivo – Criação e evolução contínua por meio da interação entre múltiplas plataformas, organizações e usuários.

Engenharia de Software Conectada

- Visão de negócio – Alinhamento técnico com as metas estratégicas da organização.
- Arquitetura modular – Estruturas que facilitam a integração e a escalabilidade.
- Práticas ágeis – Desenvolvimento colaborativo para respostas rápidas às mudanças do ecossistema.
- Pilares do Sucesso Moderno – Interoperabilidade e evolução contínua.

Compreendendo o ecossistema e o negócio digital

- O ecossistema digital de negócios integra diversos atores e tecnologias em uma rede colaborativa voltada para a geração contínua de valor.
- Sua sustentação depende de arquiteturas modulares e interoperáveis que permitem a escalabilidade e a rápida adaptação às demandas do mercado.

Infraestrutura de um ecossistema digital de negócios:

Módulo	Representação visual na imagem a seguir
Web Content Providers	Os ícones que lembram pastas e símbolos digitais (como o ícone azul à direita).
Cloud Service Providers	Os blocos empilhados e brilhantes (azul ou cinza no topo).
Electronic Trading	Os nódulos de destaque (vermelho) ou pts. de conexão rápida.
Enterprises	O prédio central e os blocos maiores com janelas.
Network Services Providers	Toda a grade de linhas e circuitos que conectam os blocos.

Principais componentes de um ecossistema digital

- O ecossistema digital é formado por um conjunto de elementos interdependentes que trabalham juntos para criar, entregar, integrar e evoluir produtos e serviços digitais.
- Esses elementos não existem isoladamente: eles cooperam, competem e se influenciam, formando um ambiente dinâmico, vivo e em constante transformação.

Principais componentes de um ecossistema digital

1. Plataformas e Infraestruturas Tecnológicas
2. Comunidades de Desenvolvedores e Parceiros
3. Usuários e Consumidores
4. Serviços Digitais, APIs e Módulos Interoperáveis
5. Regras de Governança e Normas
6. Dados e Fluxos de Informação
7. Ferramentas de Desenvolvimento e Suporte
8. Aplicações, Produtos e Serviços Criados Dentro do Ecossistema
9. Comunidade de Suporte, Feedback e Inovação
10. Mecanismos de Monetização e Sustentabilidade

Gestão do desenvolvimento ágil em ecossistemas digitais

A gestão do desenvolvimento de software em ecossistemas digitais exige uma abordagem capaz de lidar com alta complexidade, interdependência entre sistemas, múltiplos atores e ritmos acelerados de inovação.

São feitas as seguintes observações:

- Papel da gestão ágil – Atuação estratégica na coordenação de equipes, plataformas, usuários, dados e serviços distribuídos, indo além de práticas operacionais.
- Fundamentos tecnológicos – Uso de arquiteturas distribuídas, APIs, microsserviços e evolução incremental das aplicações.
- Colaboração e stakeholders – Participação ativa de fornecedores, parceiros, desenvolvedores e usuários finais na construção e evolução do software.
- Tomada de decisão orientada a dados – Transparência, uso de métricas, validação contínua e sincronização entre equipes (backlogs compartilhados e cadências).
- Agilidade como competência organizacional – Capacidade de adaptação contínua do software ao ecossistema digital, preservando compatibilidade, segurança e geração de valor.

Ecossistemas distribuídos e inteligentes

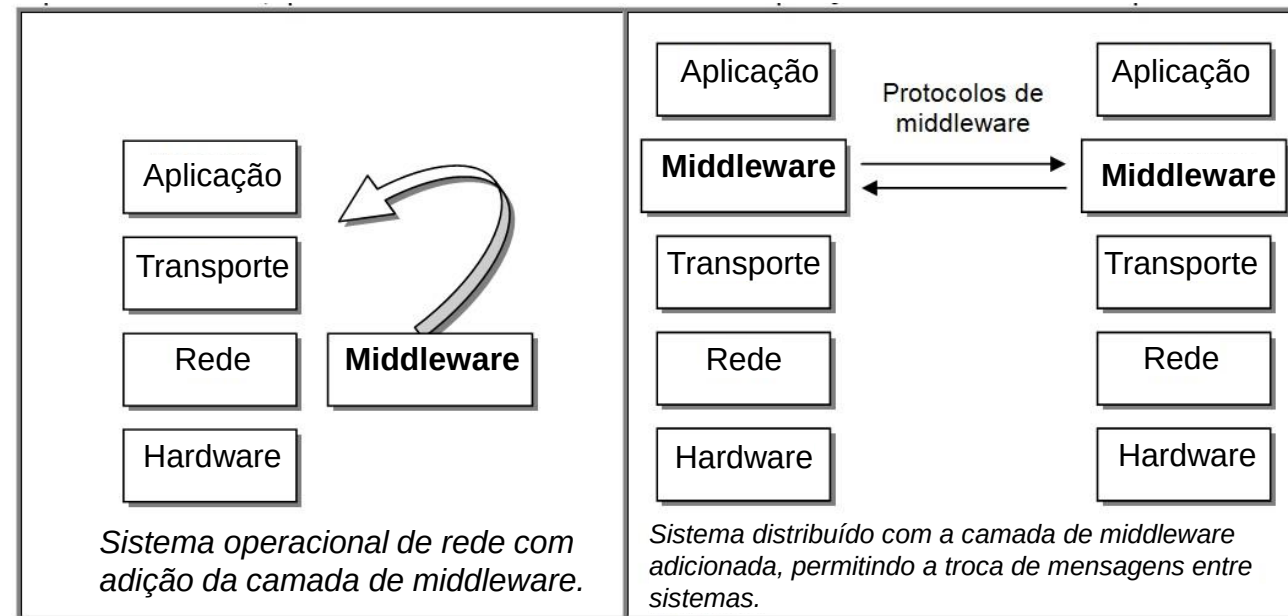
- Arquitetura distribuída – São diversos sistemas computacionais, independentes de seus ambientes operacionais que trocam mensagens, capazes de operar em larga escala, atender múltiplos usuários simultaneamente e integrar-se a sistemas externos. Formados basicamente por clusters de computadores.
 - É o projeto de infraestrutura do ecossistema digital.
- SOA (Service-Oriented Architecture) – A Arquitetura Orientada a Serviços introduz o conceito de modularidade baseada em serviços reutilizáveis, como Integração de sistemas heterogêneos, Orquestração de serviços, Gerência de comunicação e Distribuição geográfica de componentes.
 - São os serviços desenvolvidos por analistas, programadores e administradores do BD.
- APIs (Application Programming Interfaces) – São microsserviços baseados em Interfaces de Programação de Aplicações (APIs).
 - Microsserviços como PayPal API, Mercado Pago / Pix, OpenAI API, Google Maps e outros.

Arquitetura distribuída e multiprocessamento

- Da arquitetura monolítica centralizada à escalabilidade global.
- O salto evolutivo foi motivado pela interoperabilidade, disponibilidade e gestão de recursos.
- Os sistemas distribuídos possuem uma estrutura que se baseia em um conjunto de computadores autônomos conectados em rede, sua memória é distribuída pelas ligações da rede, o foco está em sistemas web, cloud e microsserviços, e tem como vantagem escalabilidade horizontal e tolerância a falhas.
- Em sistemas distribuídos, o middleware é o principal componente e está localizado exatamente entre as camadas de Aplicação e Transporte de um sistema operacional de rede (SOR).

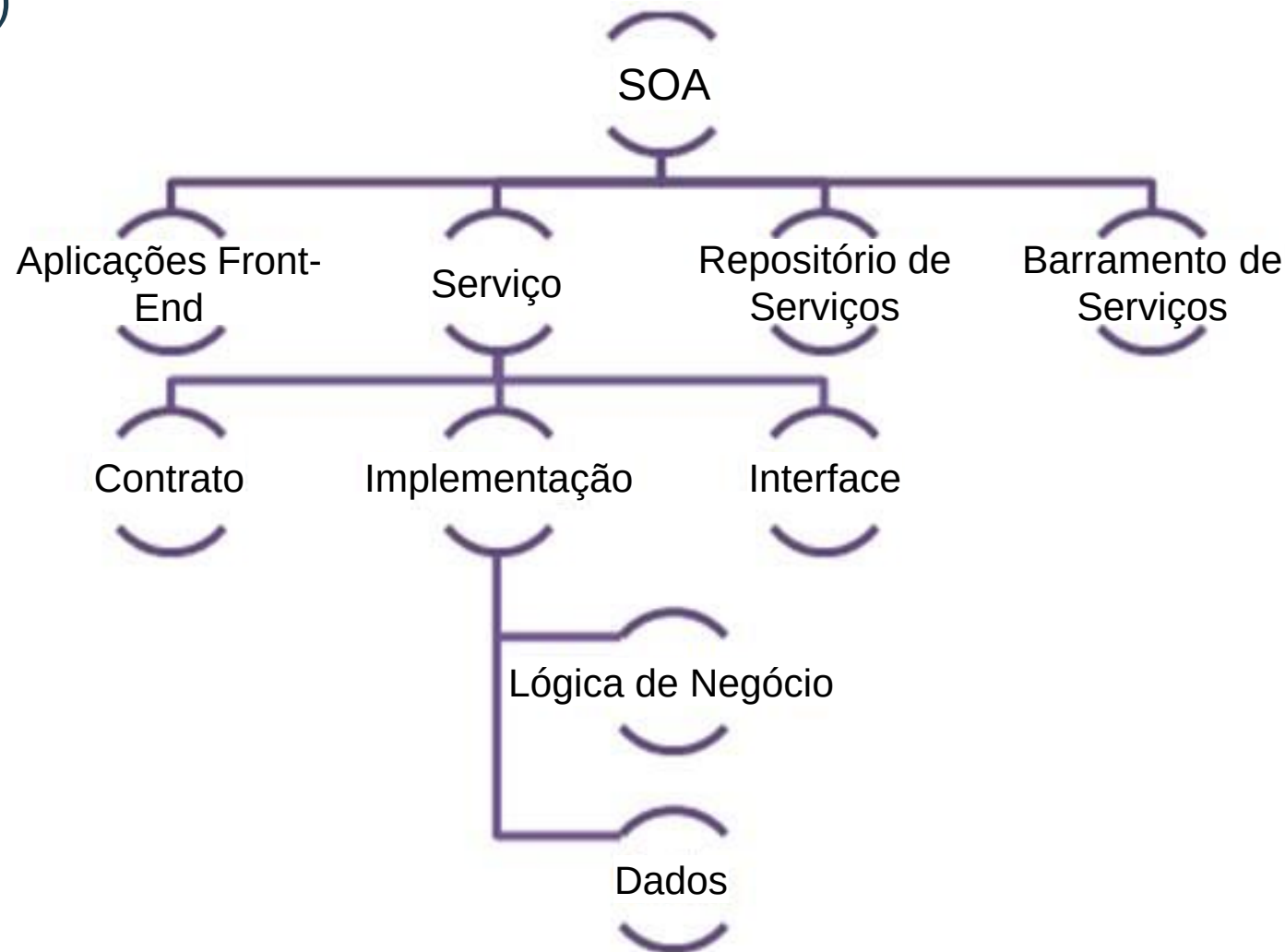
**Implementação da camada de rede
middleware para um sistema distribuído.**

Fonte: Moreno (2020).



SOA (Service-Oriented Architecture)

- A SOA (Arquitetura Orientada a Serviços) introduz o conceito de modularidade baseada em serviços reutilizáveis.
- A SOA contribui também para atributos de qualidade como manutenibilidade, reusabilidade, escalabilidade e confiabilidade, ao adotar princípios como baixo acoplamento, alta coesão e contratos bem definidos, consolidando-se como base para arquiteturas distribuídas modernas.
- A arquitetura é organizada em camadas funcionais.



Elementos de uma Arquitetura Orientada a Serviços.

Fonte: <https://arquivo.devmedia.com.br/REVISTAS/es/imagens/31/4/image001.jpg>.

Interatividade

Indique **Verdadeiro** (V) ou **Falso** (F) para as afirmações a seguir, sobre ecossistemas digitais e engenharia de software conectada, e assinale a alternativa correta.

- I. A engenharia de software conectada prioriza o alinhamento técnico com os objetivos de negócio da organização.
- II. Arquiteturas modulares dificultam a integração e reduzem a escalabilidade dos sistemas.
- III. As práticas ágeis favorecem a colaboração e a adaptação contínua às mudanças do ecossistema digital.
- IV. Em ecossistemas digitais, o software atua como parte de uma rede interconectada de plataformas, serviços e usuários.

- a) F – F – V – F.
- b) F – V – F – V.
- c) V – F – V – V.
- d) V – V – F – V.
- e) V – V – V – F.

Resposta

Indique **Verdadeiro** (V) ou **Falso** (F) para as afirmações a seguir, sobre ecossistemas digitais e engenharia de software conectada, e assinale a alternativa correta.

- I. A engenharia de software conectada prioriza o alinhamento técnico com os objetivos de negócio da organização.
- II. Arquiteturas modulares dificultam a integração e reduzem a escalabilidade dos sistemas.
- III. As práticas ágeis favorecem a colaboração e a adaptação contínua às mudanças do ecossistema digital.
- IV. Em ecossistemas digitais, o software atua como parte de uma rede interconectada de plataformas, serviços e usuários.

- a) F – F – V – F.
- b) F – V – F – V.
- c) **V – F – V – V.**
- d) V – V – F – V.
- e) V – V – V – F.

APIs (Application Programming Interfaces)

- A evolução das SOAs para microserviços e computação em nuvem consolidou as APIs como principais meios de integração entre sistemas.
- APIs expõem funcionalidades de forma padronizada, segura e modular, permitindo a comunicação entre aplicações, plataformas, dados e serviços externos.
- Atuam como elementos centrais dos ecossistemas digitais, conectando empresas, desenvolvedores e usuários e viabilizando inovação por recombinação de serviços.
- Integradas a arquiteturas distribuídas, suportam escalabilidade, resiliência e desempenho, com adaptação dinâmica à demanda.
- Exigem governança arquitetural, incluindo versionamento, contratos estáveis e padrões, garantindo a evolução contínua e interoperabilidade.



Veja como é fácil. Faça um teste prático.

Caso: Inclusão da API de clima

Este caso se refere a uma chamada de API para receber acesso aos vários dados como: atual clima e previsões, dados meteorológicos, visão geral do tempo e outros recursos. Consulte a referência bibliográfica abaixo:

OpenWeathern®. API de clima. Weather API – OpenWeatherMap®, 2025. Disponível em:

<https://openweathermap.org/ap>. Acesso em: 31 jan. 2026.

Customização de serviços em ecossistemas digitais

Conceito e importância

- Estratégia essencial para atender a demandas específicas de usuários, aplicações e contextos operacionais.
- Diferencial técnico e competitivo em ambientes distribuídos e dinâmicos, especialmente com microsserviços.

Camadas de customização

- Parametrização de APIs e configuração de endpoints de uma conexão (GET, POST, PUT/PATCH e DELETE).
- Personalização da lógica de negócio e da experiência do usuário (UX).
- Ajustes dinâmicos de comportamento conforme contexto e demanda.

Ferramentas de suporte

- API Gateway – ponto único de entrada, controle de acesso, roteamento e segurança.
- Service Mesh – gerenciamento da comunicação entre serviços (observabilidade, segurança, resiliência).
- Plataformas de orquestração – automação e coordenação de serviços e infraestrutura.

Customização Ágil e Infraestrutura como Código (IaC)

Customização e Ciclos Ágeis

A adaptação contínua é viabilizada por:

- **Feedback e Entregas Curtas:** Ciclos rápidos que permitem ajustes incrementais.
- **Equipes Multidisciplinares:** Colaboração para alinhar tecnologia e objetivos de negócio.
- **Foco em Produto:** Criação de serviços modulares que podem ser recombinaados e evoluídos conforme a demanda.

Infraestrutura como Código (IaC)

A transição do manual para o automático:

- O Conceito: Substitui painéis e cliques manuais por arquivos de configuração rastreáveis.
- Vantagens: Provisionamento automático de servidores, redes e bancos de dados com maior segurança, velocidade e previsibilidade.
- Resiliência: Permite o monitoramento contínuo e a rápida recuperação/adaptação do ecossistema.

Customização em ecossistemas digitais

Da modelagem à entrega personalizada, a customização em ecossistemas digitais compreende 7 etapas de implementação:

1. Modelagem Modular (DDD) – Alinhamento entre técnica e negócio. Cada serviço reflete uma funcionalidade real (ex.: "Pagamento", "Estoque").
2. APIs Padronizadas (REST) – Garantia de que o sistema "fala" com o mundo de forma interoperável.
3. Configuração Dinâmica – Separação entre código e configuração. Uso de Feature Toggles para ligar/desligar funções sem novas implementações.
4. Orquestração Inteligente – API Gateway: Ponto único de entrada e segurança.
 1. Service Mesh: Controle fino de tráfego e resiliência entre serviços.
5. Entrega Contínua (CI/CD) – Automação total com GitHub Actions. Uso de Docker (pacotes) e Kubernetes (escalabilidade) para rapidez e segurança.
6. Monitoramento e Feedback – Visibilidade em tempo real com Prometheus (coleta de dados) e Grafana (dashboards visuais).
7. Personalização e Segurança – Uso de OAuth2 e JWT para acessos seguros e experiências baseadas no perfil/contexto do usuário (ex.: "Entrar com Google").

MF&T: Desenvolvimento ágil em ecossistemas digitais

- Ecossistemas digitais são complexos, com grande volume de mudanças e forte interdependência entre sistemas.
- A agilidade deve coexistir com estabilidade, qualidade e governança.
- MF&T são amplamente aplicados em e-commerce, serviços financeiros, streaming e plataformas globais.
- Frameworks escaláveis, automação e arquiteturas distribuídas viabilizam entrega contínua de valor em ambientes interconectados.

Agilidade como competência estratégica

- Suporta interoperabilidade, escalabilidade e rápida adaptação tecnológica.
- Responde a mudanças de mercado, novos requisitos e integrações complexas.

Principais métodos utilizados

- Scrum – Entregas incrementais em sprints curtos e coordenação entre squads.
- Kanban – Gestão contínua do fluxo de trabalho e demandas recorrentes.
- SAFe – Escalabilidade do ágil em organizações com múltiplos times e produtos.
- DevOps – Integração entre desenvolvimento e operações com automação e monitoramento contínuo.

Ferramentas em ecossistemas digitais

Base tecnológica do desenvolvimento ágil

- Sustentam colaboração, automação, integração e operação contínua.
- Fortalecem a governança tecnológica e a confiabilidade dos serviços.

Principais categorias de ferramentas

- Gerenciamento Ágil – Jira, Trello, Azure DevOps: Planejamento e acompanhamento.
- CI/CD – GitHub Actions, Jenkins, GitLab CI: Builds, testes e deploy automatizados.
- Containers & Orquestração – Docker, Kubernetes: Escalabilidade e gestão de microsserviços.
- Monitoramento – Prometheus, Grafana, ELK: Métricas, logs e observabilidade.
- APIs – Swagger (documentação e design), Postman (desenvolvimento e teste), Kong (segurança/gateway): Design, testes e governança de APIs.

Técnicas práticas e benefícios em ecossistemas digitais

Técnicas essenciais

- TDD, CI e CD – Qualidade integrada e entregas frequentes.
- Infraestrutura como Código (IaC) – Ambientes padronizados e automatizados.
- APIs First – Alinhamento entre equipes e interoperabilidade.
- Arquitetura de Microsserviços – Modularidade, escalabilidade e evolução contínua.
- Pair Programming, Code Review e MVP – Qualidade, aprendizado e validação rápida.

Principais benefícios

- Escalabilidade horizontal e resiliência.
- Integração via APIs padronizadas.
- Adaptação rápida a mudanças.
- Colaboração entre equipes multidisciplinares.
- Entrega contínua de valor em ambientes digitais dinâmicos.

Interatividade

Quando MF&T (Métodos, Frameworks e Técnicas) são aplicados ao desenvolvimento ágil em ecossistemas digitais, a agilidade assume o papel de competência estratégica. Considerando as características desses ambientes, assinale a alternativa que melhor expressa essa função da agilidade.

- a) Eliminar a necessidade de governança para acelerar a entrega de software.
- b) Priorizar exclusivamente a velocidade de desenvolvimento em detrimento da qualidade.
- c) Reduzir a complexidade dos ecossistemas por meio de sistemas monolíticos.
- d) Restringir a integração entre sistemas para garantir maior estabilidade.
- e) Viabilizar interoperabilidade, escalabilidade e adaptação rápida às mudanças tecnológicas e de mercado.

Resposta

Quando MF&T (Métodos, Frameworks e Técnicas) são aplicados ao desenvolvimento ágil em ecossistemas digitais, a agilidade assume o papel de competência estratégica. Considerando as características desses ambientes, assinale a alternativa que melhor expressa essa função da agilidade.

- a) Eliminar a necessidade de governança para acelerar a entrega de software.
- b) Priorizar exclusivamente a velocidade de desenvolvimento em detrimento da qualidade.
- c) Reduzir a complexidade dos ecossistemas por meio de sistemas monolíticos.
- d) Restringir a integração entre sistemas para garantir maior estabilidade.
- e) Viabilizar interoperabilidade, escalabilidade e adaptação rápida às mudanças tecnológicas e de mercado.

Referências

- ALBIN, S. T. *The art of software architecture: design methods and techniques*. Indianapolis: John Wiley, 2003.
- PMBOK®. *Guia do conhecimento em gerenciamento de projetos – Guia PMBOK®*. 7. ed. Atlanta, USA: Project Management Institute (PMI), Inc. / Project Management Body of Knowledge (PMBOK®), 2021.
- PMBOK®. *Um guia do conhecimento em gerenciamento de projetos – Guia PMBOK®*. 6. ed. Atlanta: Project Management Institute (PMI), Inc. / Project Management Body of Knowledge (PMBOK®), 2017.
- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH 2016. (968 p.)
- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de software: uma abordagem profissional*. 9. ed. Porto Alegre: AMGH, 2021.
- SOMMERVILLE, Ian. *Software engineering*. 10a. ed. São Paulo: Pearson Education, 2021.

ATÉ A PRÓXIMA!